

RELAYSPEC: ADAPTING FROZEN SPECULATIVE DRAFTERS WITH LINEAR MAPS

Anonymous authors

Paper under double-blind review

ABSTRACT

Speculative drafters that use a target model’s hidden states are trained for a particular target. A service may activate a LoRA adapter or switch to a model of a different size or family, changing the hidden states supplied to the drafter. RelaySpec reuses the pretrained drafter by learning one linear map for each target feature stream while keeping the drafter transformer frozen. The maps fold into its existing fusion projection before inference. For LoRA-fine-tuned targets, we train the maps on target-generated responses. For cross-transfer, we train them to reconstruct the original target’s features. We evaluate mathematical reasoning, circuit design, coding, and chat workloads. On some LoRA-fine-tuned targets, RelaySpec improves DFlash throughput by up to 85% over the unadapted drafter. In Qwen3-4B to Qwen3-8B cross-transfer, it reaches up to 100.2% of native-drafter throughput on MATH and 99.4% on GSM8K. The same approach generalizes to EAGLE-3 drafters, where it improves throughput by 46% to 106% over the unadapted version on LoRA-fine-tuned targets.

1 INTRODUCTION

As Large Language Models (LLMs) continue to advance, efficient inference remains a challenge. Autoregressive decoding generates one token at a time. This inherent seriality creates a major performance bottleneck: decoding is slow, memory-bound, and fails to fully utilize modern GPUs (Cai et al., 2024).

Speculative decoding accelerates a language model, called the *target*, using a smaller model or prediction module, called the *drafter*. The drafter proposes several candidate tokens, and the target checks them in parallel. Verification keeps an accepted prefix and replaces the first rejected proposal with a correction. Appropriate acceptance and correction rules preserve the target’s output distribution (Leviathan et al., 2023; Chen et al., 2023). Speedup depends primarily on the length of the accepted prefix in each proposed block, together with the time spent drafting and verifying that block.

Some drafters also use the target’s hidden-state representations. These representations provide contextual information that can help the drafter propose tokens the target will accept. Medusa adds parallel prediction heads to the target’s final hidden states (Cai et al., 2024). The EAGLE family uses target features to guide autoregressive drafting (Li et al., 2024b;c; 2025). DFlash instead uses a block-diffusion drafter to predict several tokens in one pass and injects fused target hidden-state representations into every layer of the drafter model (Chen et al., 2026).

This dependence complicates reuse of the drafter model when a service changes its target’s size or family, or LoRA-fine-tunes it (Hu et al., 2022). The new target can produce different hidden states and preferred continuations, and changed feature dimensions may be incompatible with the existing drafter. Training a new drafter model requires target-specific supervision, optimization of the drafter’s weights, and validation (Li et al., 2025; Chen et al., 2026). Repeating this process across target variants increases training and maintenance costs. Can we instead reuse an existing drafter by adapting only its target-feature inputs?

We introduce RelaySpec, a method for reusing a pretrained DFlash drafter (Chen et al., 2026) after its target changes. Section 4 explains the parts of DFlash that RelaySpec uses. The new target supplies hidden states from five layers. A separate linear map transforms each stream before the

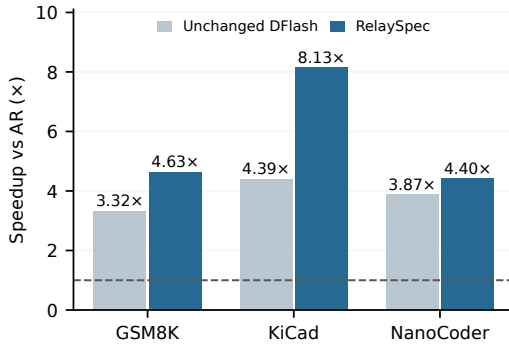


Figure 1: LoRA-fine-tuned Qwen3-4B targets. Mean single-request speedup over autoregressive decoding on 128 prompts.

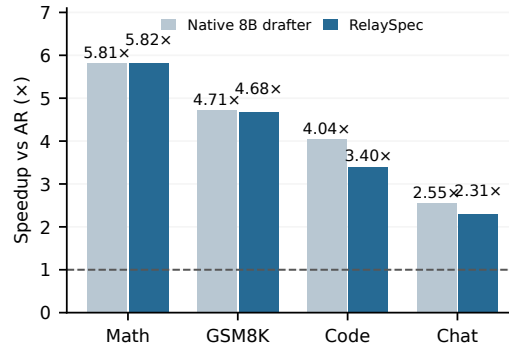


Figure 2: Qwen3-4B drafter reused with Qwen3-8B. Mean single-request speedup over autoregressive decoding on 128 prompts.

drafter’s original fusion projection combines them. The resulting context conditions the drafter transformer, which predicts a block of candidate tokens.

During adaptation, only the five maps are trained. The target, original fusion projection, drafter transformer, normalization layers, and the embeddings and output head selected for the model pair remain frozen. A map can be rectangular when the target and drafter expect different feature dimensions. Before inference, we multiply each map into its corresponding fusion block, a composition of linear operations. This produces one effective projection, so inference needs no separate mapping operation.

RelaySpec uses training signals that match the change in the feature interface. For a LoRA-fine-tuned target, identity-initialized maps preserve the existing interface and are trained to improve the accepted prefix. For cross-model transfer, the maps reconstruct the original target’s features and fused context from the new target’s features. Section 4 gives both procedures.

Figures 1 and 2 show the measured throughput in these two settings relative to autoregressive decoding.

We analyze the learned maps by compressing their corrections with SVD, scaling the correction strength, comparing the corrections with LoRA-induced feature changes, and interpolating between token-trained and MSE-trained maps. We measure how these interventions change acceptance and throughput.

2 PRELIMINARIES

The *original target* is the model used to train the available drafter. The *new target* is the model we want to accelerate. For cross-transfer, the original/reference target supplies reference features for training the maps and we compare against the native drafter. For a LoRA-fine-tuned target, we compare with the unadapted drafter: the released checkpoint used without maps.

3 RELATED WORK

Adapting and reusing drafters. EDA and RelaySpec both use target-specific training data, but EDA separates the drafter into shared and target-specific components and trains the lightweight private component, whereas RelaySpec trains five maps at DFlash’s hidden-state input and leaves the drafter transformer unchanged (Lin et al., 2026). Online Speculative Decoding continually updates one or more drafters from observed serving queries, and OmniDraft adds an online n-gram cache, hybrid distillation, and vocabulary conversion, whereas RelaySpec trains its maps offline and performs no serving-time training (Liu et al., 2024; Ramakrishnan et al., 2025). PARD trains a target-independent parallel drafter for a family of targets, and PARD-2 trains one drafter to support

target-independent and target-conditioned modes, whereas RelaySpec starts from an already trained target-conditioned DFlash drafter and changes only its feature interface (An et al., 2026a;b).

Adapting pretrained drafters through target features. SD^2 is the closest method-level comparison because it also conditions a pretrained drafter on target hidden states (Berdoz et al., 2026). Its main method injects a learned steering signal into every feed-forward layer and fully fine-tunes the autoregressive drafter, whereas RelaySpec maps the five feature streams already used by DFlash and keeps its transformer frozen. SD^2 also tests a frozen drafter, but its signal is still injected within every feed-forward layer, whereas RelaySpec places the maps before the existing fusion projection and folds them into that projection before inference. RepSpec adds linear structure during drafter training and merges it into the model for inference, whereas RelaySpec trains maps on the feature inputs of an existing drafter and folds them into its fusion projection without updating its transformer (Huo et al., 2026).

4 METHOD

RelaySpec reuses a pretrained drafter after its target changes through LoRA fine-tuning or cross-transfer. It trains maps on the target features supplied to the drafter while keeping the target and drafter weights frozen. We describe the DFlash interface, how the maps are trained, and how the same construction extends to EAGLE-3.

4.1 DFLASH AND THE FEATURE MAPS

DFlash begins each block with the last token already committed by the target, called the anchor (Chen et al., 2026). The drafter receives this known token followed by 15 masked positions in the released Qwen3-4B configuration. It predicts the next 15 tokens in one pass; the anchor itself is not a proposal. The target checks the proposals in parallel.

DFlash also reads hidden states from five fixed target layers. Its original fusion projection combines those states, applies RMSNorm, and supplies the resulting context as keys and values to every drafter transformer layer (Chen et al., 2026; Zhang & Sennrich, 2019). The anchor and masked positions provide the queries. Only target states from before the anchor are visible. Let $K = 5$, and let $h_{i,t}^{(o)} \in \mathbb{R}^{d_r}$ and $h_{i,t} \in \mathbb{R}^{d_t}$ be features from layer i and token t of the original and new targets. The original fusion block $F_i \in \mathbb{R}^{d_d \times d_r}$ projects to drafter width d_d . RelaySpec places a bias-free map $W_i \in \mathbb{R}^{d_r \times d_t}$ before each block. The original and adapted contexts are

$$c_t^{(o)} = N\left(\sum_{i=1}^K F_i h_{i,t}^{(o)}\right), \quad \hat{c}_t = N\left(\sum_{i=1}^K F_i W_i h_{i,t}\right), \quad (1)$$

where $N : \mathbb{R}^{d_d} \rightarrow \mathbb{R}^{d_d}$ is the drafter’s frozen RMSNorm. The same map is applied at every token position. Only the W_i are trained. The target, fusion blocks, RMSNorm, drafter transformer, and selected token embeddings and output head remain frozen.

Before inference, we multiply each map into its fusion block and join the products:

$$G = [F_1 W_1 \cdots F_K W_K] \in \mathbb{R}^{d_d \times K d_t}, \quad \hat{c}_t = N(G[h_{1,t}; \dots; h_{K,t}]). \quad (2)$$

Thus one projection computes the mapped context without a separate mapping operation. The original normalization remains afterward. The linear maps can change feature width and still fold into this projection. Section 5.3.3 compares them with a nonlinear alternative.

4.2 TRAINING THE MAPS

LoRA-fine-tuned targets. A LoRA-fine-tuned target changes the target but leaves its feature width unchanged (Hu et al., 2022). We start each square map at $W_i = I$, preserving the unchanged drafter’s initial computation with that LoRA-fine-tuned target. We save target-generated responses and sample anchor positions within them. At each sampled position, the saved token is the known anchor and the following saved tokens provide up to 15 labels for the masked predictions. The anchor receives no loss. Since verification stops at the first wrong proposal, we train on the correct prefix and the first wrong proposal, a rule called accept-until-fail (AUF) (Yang & Li, 2026).

For block b and proposal position j , let q_{bj} be the drafter’s token distribution, y_{bj} the saved target token, and m_{bj} a mask for valid response labels. The loss is

$$u_{bj} = \mathbf{1} \left[\arg \max_v q_{bj}(v) = y_{bj} \text{ or } m_{bj} = 0 \right], \quad s_{bj} = \text{stopgrad} \left(\prod_{k < j} u_{bk} \right),$$

$$\mathcal{L}_{\text{AUF}} = - \frac{\sum_{b,j} m_{bj} s_{bj} \log q_{bj}(y_{bj})}{\sum_{b,j} m_{bj} s_{bj}}. \quad (3)$$

The product tests only earlier positions, so the first error contributes to the loss. We recompute the mask from current predictions and hold it fixed during differentiation. Gradients pass through the frozen drafter to the maps. Each microbatch loss is normalized by its supervised positions before averaging across workers and accumulation steps.

Cross-model transfer. When the new target is a different model, its hidden states may differ in width and content. We initialize the maps with independent Xavier-uniform values (Glorot & Bengio, 2010). We generate a response with the new target, then run both targets on the same saved text to collect paired features. The original target supplies training references but is absent during adapted inference. When tokenizers differ, feature positions are paired only at the same text prefix.

For example b , let S_b contain selected valid prompt and response positions. At position t , h_{bti} and $h_{bti}^{(o)}$ are paired features, and $c_{bt}^{(o)} = N(\sum_i F_i h_{bti}^{(o)})$ is the original context. We train the maps to match both the individual features and their fused context:

$$D(u, v) = \frac{\|u - v\|_2^2}{\|v\|_2^2 + 10^{-6}}, \quad (4)$$

$$\mathcal{L}_{\text{MSE}} = \frac{1}{B} \sum_{b=1}^B \frac{1}{|S_b|} \sum_{t \in S_b} \left[\frac{1}{K} \sum_{i=1}^K D(W_i h_{bti}, h_{bti}^{(o)}) + D(\hat{c}_{bt}, c_{bt}^{(o)}) \right]. \quad (5)$$

The feature and context terms have equal weight. Squared errors are divided by the corresponding original vector’s squared norm plus a small constant. We exclude padding and average positions within each example before averaging examples. Unlike AUF, this loss does not require gradients through the drafter transformer.

4.3 EXTENSION TO EAGLE-3

The same mapping idea applies to EAGLE-3, which reads three target feature streams and drafts tokens autoregressively (Li et al., 2025). We use $K = 3$ maps before its existing feature projection, retain the original prediction network, and fold the maps into that projection as in Equation 2. For a LoRA-fine-tuned target, identity-initialized maps receive token supervision along EAGLE-3’s recurrent path. At each sampled anchor, the loss covers up to eight successive predictions through the first wrong token. A target token outside EAGLE-3’s reduced vocabulary stops the path without a loss term. For cross-transfer, Xavier-initialized maps use the paired-feature loss in Equation 5.

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

Models. DFlash experiments reuse the Qwen3-4B drafter (Chen et al., 2026; Yang et al., 2025). EAGLE-3 experiments use its published checkpoints (Li et al., 2025). Their draft transformers remain frozen.

Targets. LoRA-fine-tuned targets are Qwen3-4B with GSM8K, KiCad, or NanoCoder adapters and Qwen3-8B with GSM8K or CodeAlpaca adapters (Hu et al., 2022; Chaudhary, 2023). Cross-transfer uses Qwen3-8B $\leftarrow$ Qwen3-4B, Qwen3-4B $\leftarrow$ Qwen3-8B, Llama3.1-8B $\leftarrow$ Qwen3-4B, and Llama3.2-3B $\leftarrow$ Llama3.1-8B (Grattafiori et al., 2024). The arrow points from the drafter’s original target to the new target.

References. As defined in Section 2, LoRA-fine-tuned results use the unadapted drafter and cross-transfer results use a native drafter. Autoregressive decoding uses the same target without a drafter.

Table 1: Single-request results on 128 prompts per workload. AR is autoregressive throughput in tokens/s. Speedup is relative to AR. The reference is unadapted for LoRA-fine-tuned targets and native for cross-transfer. Fit time is the selected map fit on two L40S GPUs and applies to every workload for that target and drafter. Bold marks RelaySpec/reference ratios at or above $1.00\times$. Protocol: Appendix A.

Target $\leftarrow$ drafter	Workload	Shared fit time (s)	AR (tok/s)	RelaySpec		Reference drafter		
				Speedup	τ	Speedup	τ	$\times$ ref
<i>LoRA-fine-tuned target. Reference: unadapted drafter.</i>								
Qwen3-4B + GSM8K LoRA	GSM8K	159.8	38.81	$4.63\times$	5.99	$3.32\times$	4.47	$1.40\times$
Qwen3-4B + KiCad LoRA	KiCad	319.3	38.42	$8.13\times$	11.26	$4.39\times$	5.99	$1.85\times$
Qwen3-4B + NanoCoder LoRA	NanoCoder	165.4	39.01	$4.40\times$	5.55	$3.87\times$	5.11	$1.14\times$
Qwen3-8B + GSM8K LoRA	GSM8K	243.6	37.89	$4.59\times$	6.35	$3.40\times$	4.76	$1.35\times$
Qwen3-8B + CodeAlpaca LoRA	CodeAlpaca	241.2	37.56	$4.32\times$	5.63	$4.07\times$	5.62	$1.06\times$
<i>Cross-transfer. Reference: native drafter.</i>								
Qwen3-8B $\leftarrow$ Qwen3-4B	Math	1494.6	37.84	$5.82\times$	7.98	$5.81\times$	8.21	$1.00\times$
	GSM8K		37.76	$4.68\times$	6.24	$4.71\times$	6.48	$0.99\times$
	Code		37.50	$3.40\times$	4.74	$4.04\times$	5.64	$0.84\times$
	Chat		35.09	$2.31\times$	2.89	$2.55\times$	3.32	$0.90\times$
Qwen3-4B $\leftarrow$ Qwen3-8B	Math	1145.4	39.42	$5.49\times$	7.66	$5.70\times$	8.09	$0.96\times$
	GSM8K		39.72	$4.27\times$	5.99	$4.46\times$	6.34	$0.96\times$
	Code		39.60	$3.30\times$	4.89	$4.34\times$	6.26	$0.76\times$
	Chat		40.48	$2.01\times$	2.83	$2.36\times$	3.36	$0.85\times$
Llama3.1-8B $\leftarrow$ Qwen3-4B	Math	1233.0	41.00	$2.97\times$	4.63	$3.17\times$	5.12	$0.94\times$
	GSM8K		41.11	$2.21\times$	3.23	$3.10\times$	4.42	$0.71\times$
	Code		40.79	$1.98\times$	3.05	$3.00\times$	4.52	$0.66\times$
	Chat		40.96	$1.48\times$	2.06	$2.79\times$	3.95	$0.53\times$

Evaluation. Single-request DFlash evaluation uses BF16 Transformers inference with greedy decoding (Wolf et al., 2020). Cross-transfer uses MATH, GSM8K, Code from LiveCodeBench, and Chat from UltraFeedback (Hendrycks et al., 2021; Cobbe et al., 2021; Jain et al., 2024; Cui et al., 2024). LoRA-fine-tuned results use held-out prompts from each adapter’s domain. Each cell has 128 prompts and a 2,048-token cap, except KiCad at 8,192. Native DFlash outputs were verified to match the adapted outputs for every single-request evaluation with a native-DFlash control.

Metrics. Speedup divides mean per-request throughput by autoregressive throughput on the same prompts. Acceptance length τ is pooled emitted tokens per verification step, including the correction token. Serving throughput divides returned tokens by workload wall time, including drain. Ablation selections use the harmonic mean of workload ratios.

5.2 MAIN RESULTS

Table 1 compares RelaySpec with the relevant reference drafter on each target and workload.

RelaySpec improves throughput on all five LoRA-fine-tuned targets, from $1.06\times$ the unadapted drafter on CodeAlpaca to $1.85\times$ on KiCad. Acceptance length also increases in every case. KiCad rises from 5.99 to 11.26 tokens per verification step.

Every cross-transfer accelerates its target over autoregressive decoding. Qwen3-8B $\leftarrow$ Qwen3-4B reaches $1.002\times$ and $0.994\times$ native throughput on Math and GSM8K, but $0.842\times$ and $0.904\times$ on Code and Chat. Transfer across model families is weaker.

Llama3.2-3B $\leftarrow$ Llama3.1-8B has no native drafter control. It reaches $1.61\times$ to $2.35\times$ autoregressive throughput (Table 2).

5.3 ABLATION STUDIES

We test map placement, training objective, nonlinearity, and supervision budget. Comparisons use matched settings within each ablation unless stated otherwise.

5.3.1 WHERE THE TRAINED PARAMETERS SIT

We compare: (a) five separate maps, with context $z = \sum_i F_i W_i h_i$; (b) a direct residual update $F + \Delta F$ to the fused projection; (c) LoRA weights in the drafter transformer; (d) one joint map

Table 2: Llama3.2-3B $\leftarrow$ Llama3.1-8B on 128 prompts per workload. Speedup is relative to autoregressive decoding. The 1,234.8-second selected fit is shared by all workload rows. Protocol: Appendix A.

Workload	Fit time (s)	AR (tok/s)	Speedup	τ
Math	1234.8	61.90	$2.35\times$	4.78
GSM8K		64.45	$2.18\times$	4.16
Code		63.09	$1.88\times$	3.44
Chat		64.73	$1.61\times$	2.97

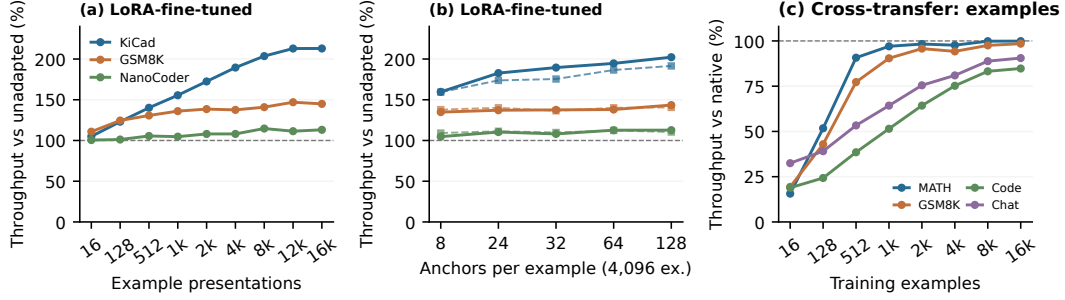


Figure 3: Supervision scaling. (a) and (b) show LoRA-fine-tuned Qwen3-4B targets relative to the unadapted drafter. Solid lines train maps and dashed lines train drafter-transformer LoRA. (c) shows Qwen3-8B $\leftarrow$ Qwen3-4B relative to the native drafter. All panels use throughput as a percentage of their stated reference; the dashed horizontal line marks 100%. Example counts range from 16 to 16,384, and panel (b) fixes 4,096 presentations while varying anchors. Appendix A.2 plots the corresponding fit times.

on $[h_1; \dots; h_5]$; and (e) rank-constrained corrections $W_i = I + B_i A_i$. We select the five separate maps because they have the highest harmonic mean across the three LoRA-fine-tuned workloads. Appendix A.1 reports all placement results.

5.3.2 WHICH OBJECTIVE TRAINS THE MAPS

We select AUF for LoRA-fine-tuned targets and reconstruction MSE for cross-transfer. AUF has the best harmonic mean in the LoRA-fine-tuned comparison, while MSE has the best one in cross-transfer. Appendix A.1 gives every result.

5.3.3 WHETHER THE MAPS NEED TO BE NONLINEAR

Using the selected five-map architecture and setting-specific objectives, we test whether nonlinearity helps. A parallel nonlinear branch changes throughput by -2.07% to $+2.97\%$ and trails the linear maps on five of seven workloads. In these single timing passes, it shows no consistent gain to justify its extra inference work. Appendix A.1 gives the full comparison.

5.3.4 HOW MUCH SUPERVISION THE MAPS NEED

Figure 3 varies training examples for the selected AUF LoRA-fine-tuning recipe and selected MSE cross-transfer recipe, and sampled anchors on LoRA-fine-tuned targets.

At 16 examples, LoRA-fine-tuned maps improve throughput by 0.5% to 10.8% over the unadapted drafter. Cross-transfer reaches only 15.6% to 32.5% of native throughput and is below autoregressive throughput on all four workloads. More examples improve all four cross-transfer workloads. In the anchor sweep, the five maps and drafter-transformer LoRA are nearly tied on KiCad at eight anchors ($+59.7\%$ and $+59.3\%$); the maps lead at 32 anchors ($+89.5\%$ and $+75.5\%$). These curves vary optimization steps along with supervision size.

Table 3: EAGLE-3 with three input maps, on 128 prompts per workload. References are unadapted for LoRA-fine-tuned targets and native for cross-transfer. Throughput is in tokens/s. Best of each pair is bold and second best is underlined. Protocol: Appendix A.

Setting	Workload	Shared fit time (s)	Tokens/s			τ	
			Reference	RelaySpec	$\times$ ref	Reference	RelaySpec
LoRA-fine-tuned Qwen3-4B, token supervision	GSM8K	163.9	<u>71.19</u>	120.35	1.69$\times$	<u>3.347</u>	5.695
	NanoCoder	184.2	<u>81.77</u>	119.30	1.46$\times$	<u>3.852</u>	5.668
	KiCad	1149.3	<u>73.82</u>	151.86	2.06$\times$	<u>3.794</u>	7.363
Qwen3-8B $\leftarrow$ Qwen3-4B, reconstruction	Math	787.9	72.18	77.99	1.08$\times$	<u>3.627</u>	3.638
	GSM8K		<u>75.58</u>	80.59	1.07$\times$	<u>3.713</u>	3.840
	Code		63.88	65.53	1.03$\times$	<u>3.351</u>	<u>3.293</u>
	Chat		<u>61.82</u>	63.41	1.03$\times$	3.186	<u>3.136</u>

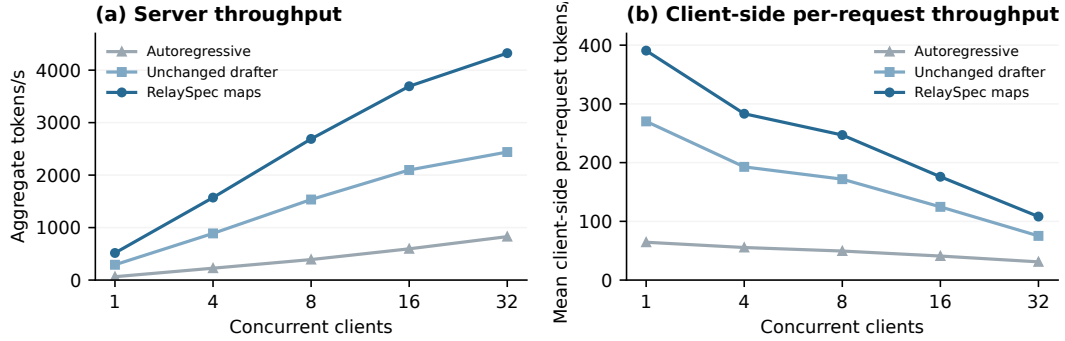


Figure 4: Three LoRA-fine-tuned Qwen3-4B targets sharing one drafter on vLLM. Each concurrency setting uses 384 requests. (a) Server aggregate throughput. (b) Mean client-side per-request throughput. Both are measured in tokens/s.

5.4 GENERALIZATION TO A SECOND DRAFTER FAMILY

We test the same input-map idea on frozen EAGLE-3 drafters. Its three streams use AUF for LoRA-fine-tuned targets and reconstruction for cross-transfer (Table 3).

AUF maps improve EAGLE-3 throughput by $1.46\times$ to $2.06\times$ over its unadapted drafter on LoRA-fine-tuned targets (Yang & Li, 2026). Reconstruction maps reach $1.03\times$ to $1.08\times$ native mean throughput on Qwen3-8B cross-transfer.

5.5 SERVING MANY TARGETS FROM ONE DRAFTER

Figure 4 tests three LoRA-fine-tuned targets sharing one target backbone and one frozen drafter on a vLLM server. Each target LoRA has its own learned five-map bank. Thus, the system is a request-level hard mixture: one target-LoRA and its matching map bank are active for an entire request. Figure 4 supplies this choice from the request’s target identifier. We also train a prompt-only hard router that selects one of the three target-LoRA/map-bank pairs before decoding. It does not mix experts token by token. Appendix A.3 gives the router training procedure, its complete measurements, and a pooled-map control.

At 32 clients, adapted aggregate throughput is 4,324 tokens/s, versus 2,440 unadapted and 828 autoregressive. Appendix A.3 gives the 16-client comparison.

Figure 5 shows Qwen3-8B $\leftarrow$ Qwen3-4B cross-transfer at 1, 8, 16, and 32 concurrent clients. RelaySpec has higher aggregate throughput than the native drafter on Math and GSM8K, and lower throughput on Code and Chat, at every measured client count. Appendix A.3 reports the 16-client measurements for all cross-transfer pairs.

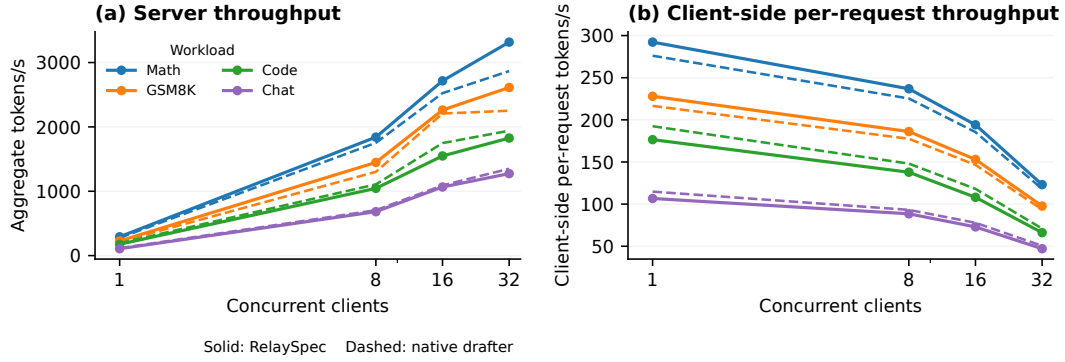


Figure 5: Qwen3-8B ← Qwen3-4B cross-transfer on vLLM. Each point uses 128 prompts per workload. Solid lines show RelaySpec and dashed lines show the native drafter. (a) Server aggregate throughput in tokens/s. (b) Client-side per-request throughput in tokens/s, computed from returned tokens and client-side request latency.

6 ANALYSIS

We analyze how the learned maps change the hidden-state context passed to the frozen drafter.

6.1 WHAT CHANGES WHEN THE INTERFACE ALREADY WORKS?

For a LoRA-fine-tuned target, identity maps recover the native DFlash interface. Write $z_0 = \sum_i F_i h_i$ for this context and $\delta = \sum_i F_i (W_i - I) h_i$ for the learned correction. We run decoding with z_0 plus either the component of δ parallel to z_0 or the component perpendicular to it.

Table 4: LoRA-target direction and geometry controls. Bold marks the best acceptance length for each task, and underline marks the second best. The geometry measurements use 1,024 positions per task and are dimensionless.

Direction intervention			LoRA-shift geometry			
Context	GSM8K τ	NanoCoder τ	Task	Correction cosine	Shuffled cosine	Radial energy
Native z_0	4.472	5.111				
Full $z_0 + \delta$	5.988	5.552	GSM8K	-0.0429	-0.0413	25.22%
Parallel only	4.466	5.114	NanoCoder	0.1248	0.1052	34.82%
Perpendicular only	<u>5.924</u>	<u>5.527</u>				17.55%
						14.88%

The parallel component returns acceptance length to the native level, while the perpendicular component recovers nearly all of the full map’s acceptance length (Table 4). Scaling the whole context by two leaves acceptance length unchanged at 5.988 tokens per verification step on GSM8K and 5.552 on NanoCoder. The useful change is mainly directional, consistent with RMSNorm suppressing common rescaling. Appendix B.1 reports every result.

Do the maps reverse the change caused by LoRA fine-tuning? We run the base and LoRA-fine-tuned targets on the same 1,024 saved token positions. At each position, we compare the map correction with the change in fused context caused by LoRA. Exact cancellation would require the correction to point in the opposite direction. We also shuffle the LoRA changes across positions while leaving the corrections fixed. This tests whether each correction is specifically aligned with the change at the same token. The mean correction cosine is -0.0429 on GSM8K and 0.1248 on NanoCoder, close to the shuffled values of -0.0413 and 0.1052 (Table 4). The maps therefore do not systematically reverse the LoRA-induced change token by token. These results support redirection of an already usable context, not literal recovery of the pre-LoRA context. Appendix B.2 defines each measurement and gives the full procedure.

Rank-128 corrections retain 95.1–99.6% of full-map latency reduction on GSM8K, while rank 256 retains 57.8–89.3% on NanoCoder. Much of the measured benefit can survive moderate-rank truncation, though timing varies (Appendix B.8.3).

6.2 WHY DOES RECONSTRUCTION HELP ACROSS MODEL SIZES?

We first compare the mapped Qwen3-8B features and fused context with those of Qwen3-4B on the same training text. MSE gives lower normalized context error and higher feature cosine than AUF or decay CE (Table 5). We then hold the prefixes and saved continuations fixed and change only the maps. On held-out MATH, Code, and Chat positions, MSE gives the lowest token cross-entropy and the longest correct prefix before the first wrong prediction. GSM8K is mixed. Thus, the recovered interface improves prediction on three of four workloads, not just the reconstruction measure. Appendix Tables 17–18 and 19–22 give the full results.

Table 5: Interface recovery on 4,096 response positions from 64 NuminaMath-derived training examples. Source cosine is reported as the minimum–maximum across the five feature streams. Error and cosine are dimensionless. Best is bold and second best is underlined.

Recipe	Peak LR	Error ↓	Source cosine min–max ↑
MSE	10^{-3}	0.136	0.944–0.985
AUF	10^{-4}	0.940	0.121–0.267
Decay CE	10^{-4}	<u>0.900</u>	<u>0.127–0.279</u>

Without retraining, maps halfway between token-trained and MSE maps improve live throughput and acceptance length on the tested MATH and Code workloads (Appendix B.5).

Does initialization explain the difference? To test whether Xavier initialization explains MSE’s advantage, we retrain from a rectangular identity, which copies the coordinates that fit the narrower interface and sets the rest to zero. All other MSE settings stay fixed, so its comparison isolates initialization. MSE acceptance length is nearly unchanged on MATH and GSM8K and slightly lower on Code and Chat, while its throughput falls on all four workloads (Table 6). AUF and decay CE improve both measurements on every workload but remain below MSE in acceptance length. Appendix Tables 25 and 26 report all endpoints.

Table 6: Initialization results on 128 prompts per task. Each cell shows Xavier → rectangular-identity initialization. Throughput is measured in tokens/s, and acceptance length τ is measured in tokens per verification step. For each dataset and metric, bold marks the best of all six values, and underline marks the second best.

Metric	Recipe	MATH	GSM8K	Code	Chat
Throughput (tokens/s)	MSE	218.75 → <u>213.70</u>	<u>172.35</u> → 166.70	125.98 → <u>119.49</u>	78.31 → <u>76.13</u>
	AUF	194.22 → 206.16	162.05 → 175.35	58.37 → 73.10	49.93 → 60.77
	Decay CE	194.69 → 203.74	163.47 → 169.72	61.56 → 76.02	51.31 → 61.11
Acceptance length τ (tokens/step)	MSE	7.973 → 7.983	6.204 → 6.217	4.690 → 4.569	2.802 → 2.747
	AUF	6.892 → 7.326	5.677 → 6.166	2.346 → 2.931	1.767 → 2.163
	Decay CE	6.947 → 7.348	5.693 → 6.073	2.449 → 3.042	1.812 → 2.183

7 CONCLUSION

RelaySpec trains linear maps on the hidden states supplied to a frozen drafter and folds them into its fusion projection for inference. The maps improve throughput on all tested LoRA-fine-tuned targets. In Qwen3-4B to Qwen3-8B cross-transfer, they approach native-drafter throughput on Math and GSM8K, with weaker results on Code, Chat, and cross-family transfers. The same input-map approach also improves a frozen EAGLE-3 drafter. Matched controls show that separate maps train better than one equally sized fused projection in the tested settings, while the analysis identifies useful changes in context direction. The drafter transformer stays shared across targets, though cross-transfer at very small data budgets and high-concurrency serving remain limited.

AI USE STATEMENT

Generative AI assisted with implementation, code review, literature search, analysis and manuscript editing.

ETHICS STATEMENT

Our method does not introduce new model capabilities or modify the learned behavior of the target model; rather, it reduces the computational overhead required to perform inference while retaining the target model as the sole authority for token verification. This work adheres to the ICLR Code of Ethics. It does not involve human subjects, sensitive data, or applications with foreseeable harmful impact. All datasets and methods are used in compliance with ethical and legal standards.

REPRODUCIBILITY STATEMENT

The appendix specifies model identifiers, feature layers, fitting settings, measurement boundaries, complete workload results and data checks. The accompanying implementation includes configurations, request records, scorer provenance and scripts that regenerate the reported assets.

REFERENCES

- Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. PARD: Accelerating LLM inference with low-cost parallel draft model adaptation. In *International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=XbOyv7iVGL>.
- Zihao An, Taichi Liu, Ziqiong Liu, Dong Li, Ruofeng Liu, and Emad Barsoum. PARD-2: Target-aligned parallel draft model for dual-mode speculative decoding. *arXiv preprint arXiv:2605.08632*, 2026b. URL <https://arxiv.org/abs/2605.08632>.
- Frédéric Berdoz, Peer Rheinboldt, and Roger Wattenhofer. Steering pretrained drafters during speculative decoding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. URL <https://ojs.aaai.org/index.php/AAAI/article/view/40255>.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 5209–5235. PMLR, 2024. URL <https://proceedings.mlr.press/v235/cai24b.html>.
- Sahil Chaudhary. Code Alpaca: An instruction-following LLaMA model for code generation. GitHub repository, <https://github.com/sahil280114/codealpaca>, 2023.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023. URL <https://arxiv.org/abs/2302.01318>.
- Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. In *International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2602.06036>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, Zhiyuan Liu, and Maosong Sun. UltraFeedback: Boosting language models with scaled AI feedback. In *International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2310.01377>.
- Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pp. 249–256. PMLR, 2010. URL <https://proceedings.mlr.press/v9/glorot10a.html>.
- Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.

- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2106.09685>.
- Feiye Huo, Jianchao Tan, Jiahao Liu, Zixu Jiang, Jiacheng Li, Jingang Wang, Xunliang Cai, and Shengli Sun. Repspec: Structural re-parameterized draft model training for speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/d70ea003729b440b89a2f958a5554c1f-Abstract-Conference.html.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024. URL <https://arxiv.org/abs/2403.07974>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023. URL <https://doi.org/10.1145/3600006.3613165>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath. Hugging Face repository, <https://huggingface.co/AI-MO/NuminaMath-CoT>, 2024a.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 28935–28948. PMLR, 2024b. URL <https://proceedings.mlr.press/v235/li24bt.html>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024c. URL <https://aclanthology.org/2024.emnlp-main.422/>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/c7b5a35ea98b62512a869c19ea7b03cb-Abstract-Conference.html.
- Luxi Lin, Zhihang Lin, Zhanpeng Zeng, Yuhao Chen, Qingyu Zhang, Jixiang Luo, Xuelong Li, and Rongrong Ji. Efficiently aligning draft models via parameter- and data-efficient adaptation. *arXiv preprint arXiv:2603.09527*, 2026. URL <https://arxiv.org/abs/2603.09527>.
- Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Online speculative decoding. *arXiv preprint arXiv:2310.07177*, 2024. URL <https://arxiv.org/abs/2310.07177>.
- Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. Omnidraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/3c2fe1417eed1c6ff9acf169617981ea-Paper-Conference.pdf.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020. URL <https://arxiv.org/abs/1910.03771>.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.

Tianjian Yang and Meng Li. Spec-AUF: Accept-until-fail training under train-inference misalignment for masked block drafters. *arXiv preprint arXiv:2607.01893*, 2026. URL <https://arxiv.org/abs/2607.01893>.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html>.

A EXPERIMENTAL PROTOCOL

Training data and selected fits. Table 1 gives the DFlash fit time for each selected target and Table 3 gives the corresponding EAGLE-3 time. All selected fits use two L40S GPUs and a global batch of eight. For Qwen3-4B LoRA-fine-tuned targets, DFlash trains five identity-initialized maps on 4,096 saved, target-generated responses from the task domain. It samples 32 valid response anchors per example, trains AUF for one epoch and uses a peak learning rate of 10^{-4} . Qwen3-8B LoRA-fine-tuned targets use the same 4,096-response, 32-anchor, one-epoch AUF recipe and learning rate. Each response is capped at 4,096 tokens during generation, and each anchor has at most 15 supervised proposal positions.

For DFlash cross-transfer, each selected fit trains five Xavier-initialized maps on 16,384 NuminaMath-derived trajectories with target-generated responses capped at 4,096 tokens (Li et al., 2024a). The original and new targets process the same saved token sequences. Reconstruction supervises 25% of prompt positions and a separate 25% of response positions. It trains for three epochs with peak learning rate 10^{-3} . Each cross-transfer fit is shared across its four evaluation workloads, so its optimizer-loop time is reported once per target-drafter pair in Tables 1 and 3, not once per dataset. EAGLE-3 uses the same selected cross-transfer data, position sampling, epoch count, and learning rate, but trains three maps. For LoRA-fine-tuned targets, EAGLE-3 trains three identity-initialized maps with AUF on 4,096 saved responses, up to 32 response anchors per example, one epoch, and peak learning rate 10^{-4} . Ablations in Section 5.3 use their stated data sizes, supervision coverage, and objectives rather than these selected recipes.

For the scaling experiments, counts denote training presentations. When a source dataset contains fewer than 4,096 available examples, examples are repeated to reach the required count.

Single-request evaluation. The DFlash measurements in Tables 1 and 2 use BF16 Transformers inference with greedy decoding and one serial request per L40S GPU (Wolf et al., 2020). Each workload has 128 fixed prompts. Generation stops at the natural end-of-sequence token or a 2,048-token response cap, except KiCad, which uses an 8,192-token cap. Thinking is disabled for Qwen3. Throughput is the arithmetic mean of each request’s returned-token count divided by its elapsed generation time. Autoregressive, reference-drafter, and mapped-drafter measurements use the same prompt cohort and decoding contract. Acceptance length is the pooled emitted-token count divided by verification steps and includes the target’s correction contribution. EAGLE-3 uses its own pinned tree settings and emitted-token counters, so its acceptance lengths are not numerically interchangeable with DFlash’s.

Concurrent serving. Serving runs use vLLM 0.28.0, BF16, greedy decoding, and fixed prompts (Kwon et al., 2023). Cross-transfer serving uses one L40S per engine, 128 prompts per workload, a 2,048-token response cap, and 16 concurrent clients for Table 13. The Qwen3-8B transfer additionally measures 1, 8, and 32 clients. The LoRA-fine-tuned shared-serving experiment in Figure 4 sends a fixed mixture of 384 requests, 128 for each of three task adapters. Its response cap is 2,048 tokens for GSM8K and NanoCoder and 8,192 for KiCad. Serving throughput divides all returned tokens by workload wall time, including drain, rather than averaging per-request rates. Native, mapped, and autoregressive runs receive the same inputs.

A.1 ARCHITECTURE AND OBJECTIVE ABLATIONS

The placement comparison uses LoRA-fine-tuned Qwen3-4B targets and the same token-supervision setting for every row. The objective comparison uses identity-initialized maps for LoRA-fine-tuned targets and Xavier-initialized maps for Qwen3-8B $\leftarrow$ Qwen3-4B cross-transfer. Its cross-transfer rows use 8,192 examples, so they are separate recipe comparisons from the selected 16,384-example result in Table 1.

Table 7: Placement ablation on LoRA-fine-tuned Qwen3-4B targets. Throughput gains are percentages over the unadapted drafter. HM is the harmonic mean of workload ratios. Best in each result column is bold and second best is underlined.

Trained object	Parameters	GSM8K (%)	KiCad (%)	NanoCoder (%)	HM
Five per-stream maps (RelaySpec)	32.8M	+ 37.6	+ 89.5	+8.1	1.376 $\times$
Single fused projection	32.8M	+32.4	+44.5	+8.2	1.265 $\times$
Draft-transformer LoRA, rank 128	36.7M	+36.4	+75.5	+ 9.8	<u>1.355</u> $\times$
Single joint map over all streams	163.8M	+24.3	+88.4	-7.0	1.244 $\times$
Rank-128 maps	3.3M	+3.2	+58.4	-28.8	0.999 $\times$
Rank-56 maps	1.4M	-21.9	+35.1	-49.3	0.751 $\times$
Rank-28 maps	0.7M	-43.1	+12.8	-62.5	0.565 $\times$

A.2 SCALING FIT TIMES

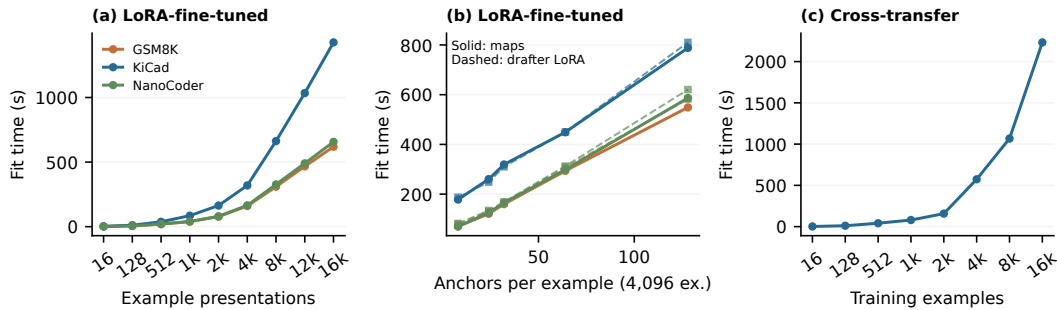


Figure 6: Fit times for the scaling measurements in Figure 3.

A.3 CONCURRENT SERVING DETAILS

Table 10 gives the 16-client point from Figure 4. It serves a fixed, equal mixture of the three LoRA-fine-tuned targets. The following subsections give the separate multi-LoRA router and pooled-map experiments. Table 13 gives all measured cross-transfer pairs at 16 clients. The Qwen3-8B $\leftarrow$ Qwen3-4B pair is also shown across client counts in Figure 5.

A.3.1 MULTIPLE LORAS AND A LEARNED ROUTER

The multi-LoRA experiment uses one BF16 Qwen3-4B target backbone, one frozen DFlash drafter, and the GSM8K, KiCad, and NanoCoder target LoRAs. Each LoRA has a separate fusion-rank-56

Table 8: Training recipes on LoRA-fine-tuned targets (top, relative to unadapted) and Qwen3-8B ← Qwen3-4B (bottom, relative to native). Entries are throughput ratios. MSE50 and MSE100 use 50% and 100% of positions. Best in each setting and column is bold and second best is underlined.

Setting	Objective	Workload			HM	
		GSM8K	KiCad	NanoCoder		
LoRA-fine-tuned	Accept-until-fail	1.376 ×	<u>1.895</u> ×	1.081 ×	1.376 ×	
	Decaying cross-entropy	<u>1.368</u> ×	1.935 ×	<u>1.031</u> ×	<u>1.353</u> ×	
	Reconstruction, MSE50	1.011×	1.191×	0.977×	1.052×	
Cross-transfer		Math	GSM8K	Code	Chat	
	Reconstruction, MSE100	<u>0.998</u> ×	0.990 ×	<u>0.828</u> ×	0.891 ×	0.921 ×
	Reconstruction, MSE50	0.999 ×	<u>0.975</u> ×	0.832 ×	<u>0.888</u> ×	<u>0.919</u> ×
	Decaying cross-entropy	0.886×	0.919×	0.406×	0.573×	0.623×
	Accept-until-fail	0.884×	0.911×	0.385×	0.558×	0.604×

Table 9: Linear maps and nonlinear branches on matched training data. Throughput is in tokens/s. Best of each pair is bold and second best is underlined.

Setting	Workload	Tokens/s		Change	τ	
		Linear	Nonlinear		Linear	Nonlinear
LoRA-fine-tuned	GSM8K	179.83	<u>177.94</u>	−1.05%	5.988	5.953
	KiCad	<u>312.47</u>	321.76	+2.97%	<u>11.256</u>	11.411
	NanoCoder	171.73	<u>169.74</u>	−1.16%	5.552	<u>5.520</u>
Cross-transfer	Math	220.26	<u>215.71</u>	−2.07%	7.984	7.980
	GSM8K	176.87	<u>174.56</u>	−1.31%	<u>6.242</u>	6.257
	Code	<u>127.61</u>	128.41	+0.63%	<u>4.739</u>	4.782
	Chat	80.93	<u>80.78</u>	−0.20%	<u>2.887</u>	2.902

Table 10: LoRA-fine-tuned shared serving at 16 concurrent clients. The fixed mixture contains 384 requests, with 128 requests per target. Aggregate and mean per-request throughput are measured in tokens/s.

Mode	Aggregate throughput	Mean per-request throughput	τ
Autoregressive	595.46	40.91	—
Unadapted drafter	2097.75	124.67	5.832
RelaySpec	3693.09	175.93	10.496

AUF map bank. Each bank is trained on 4,096 unique examples from its domain, with a 4,096-response-token cap, eight sampled anchors per response, 15 proposal labels per anchor, and 2,000 optimizer updates. The maps use random- A /zero- B initialization, AdamW at 10^{-4} , 100 warmup steps followed by cosine decay, and global batch size 8 on two L40S GPUs. The target, drafter transformer, normalization layers, embeddings, and output head remain frozen. The three fits take 3.83, 10.51, and 4.54 minutes for GSM8K, KiCad, and NanoCoder, respectively.

In the explicit-ID condition, the request specifies which target LoRA is active and selects its matching map bank. For the learned condition, a frozen prompt-only hard router selects one pair before generation. It is a multinomial logistic-regression classifier over word unigram and bigram TF-IDF features. The router trains on 4,096 user prompts per domain with 32,768 maximum features, $C = 1$, and L-BFGS. It fits in 0.944 CPU seconds and reaches 99.740% held-out source-domain accuracy. The selected pair remains fixed throughout a response. It is therefore a hard request-level mixture of LoRA/map experts, rather than a token-level mixture.

Table 11: Complete 12-client multi-LoRA map-bank results. Each repeat serves 384 requests, with 128 requests per target LoRA. Speedup is relative to the unadapted drafter.

Selection	Repeat	Speedup	τ
Explicit target ID	0	$1.3176\times$	7.678
Explicit target ID	1	$1.3288\times$	7.678
Learned hard router	0	$1.3212\times$	7.680
Learned hard router	1	$1.3243\times$	7.680

A.3.2 ONE POOLED MAP ACROSS THE THREE LoRAs

We also train one fusion-rank-56 AUF map bank on the union of the three domains. It uses the same per-domain examples, response cap, anchors, labels, initialization, and optimizer settings as the separate banks. Domain-homogeneous batches alternate round robin. Each domain receives 2,000 updates, so the shared optimizer runs for 6,000 updates and sees 48,000 response presentations. The pooled fit takes 27.12 minutes on two L40S GPUs. Table 12 gives all measurements under explicit target IDs and the learned router. The autoregressive and separate-bank controls were inherited without rerunning from the corresponding matched serving measurements.

Table 12: Complete 12-client pooled-map results for the three-LoRA mixture. Aggregate throughput is measured in tokens/s. Fit time is measured in minutes on two L40S GPUs. A dash denotes a configuration with no mapper fit.

Selection	Repeat	Configuration	Aggregate throughput (tokens/s)	τ	Speedup vs. autoregressive	Fit time (min)
Explicit target ID	0	Autoregressive	195.39	—	$1.0000\times$	—
	0	Unadapted drafter	1273.98	5.8553	$6.5200\times$	—
	0	Three map banks	1678.61	7.6782	$8.5909\times$	3.83 / 10.51 / 4.54
	0	One pooled map	1671.89	7.6170	$8.5565\times$	27.12
	1	Autoregressive	194.40	—	$1.0000\times$	—
	1	Unadapted drafter	1273.32	5.8553	$6.5500\times$	—
	1	Three map banks	1691.96	7.6782	$8.7036\times$	3.83 / 10.51 / 4.54
Learned hard router	1	One pooled map	1673.98	7.6170	$8.6111\times$	27.12
	0	Autoregressive	195.45	—	$1.0000\times$	—
	0	Unadapted drafter	1280.52	5.8560	$6.5518\times$	—
	0	Three map banks	1691.87	7.6798	$8.6565\times$	3.83 / 10.51 / 4.54
	0	One pooled map	1682.03	7.6188	$8.6061\times$	27.12
	1	Autoregressive	195.59	—	$1.0000\times$	—
	1	Unadapted drafter	1285.61	5.8560	$6.5729\times$	—
	1	Three map banks	1702.59	7.6798	$8.7048\times$	3.83 / 10.51 / 4.54
	1	One pooled map	1695.52	7.6188	$8.6687\times$	27.12

B COMPLETE ANALYSIS PROBE RESULTS

This appendix reports the complete results for every probe cited in Section 6. It also reports several completed interventions that are not discussed in the main text. All results come from the Phase 1 and Phase 2 experimental reports. They were checked against the corresponding Markdown probe reports while preparing this appendix.

Table 13: Cross-transfer serving at 16 concurrent clients. Aggregate throughput is measured in tokens/s. The higher throughput is bold and the lower is underlined.

Target $\leftarrow$ drafter	Workload	Native (tokens/s)	RelaySpec (tokens/s)	RelaySpec/native
Qwen3-8B $\leftarrow$ Qwen3-4B	Math	2524.31	2716.26	1.076 $\times$
	GSM8K	<u>2207.43</u>	2262.03	1.025 $\times$
	Code	1748.19	<u>1548.47</u>	0.886 $\times$
	Chat	1091.14	<u>1065.93</u>	0.977 $\times$
Qwen3-4B $\leftarrow$ Qwen3-8B	Math	4482.42	<u>3710.76</u>	0.828 $\times$
	GSM8K	3160.99	<u>3078.87</u>	0.974 $\times$
	Code	2840.15	<u>2002.79</u>	0.705 $\times$
	Chat	1888.69	<u>1431.21</u>	0.758 $\times$
Llama3.1-8B $\leftarrow$ Qwen3-4B	Math	1894.99	<u>1046.94</u>	0.553 $\times$
	GSM8K	1354.50	<u>853.13</u>	0.630 $\times$
	Code	1296.83	<u>636.97</u>	0.491 $\times$
	Chat	1440.04	<u>524.40</u>	0.364 $\times$

B.1 LORA-FINE-TUNED TARGETS: DIRECTION AND SCALE

For each token position, let z_0 be the fused context without the learned maps and let δ be their learned correction. The dynamic intervention constructs $z_0 + \delta_{\parallel}$ or $z_0 + \delta_{\perp}$ before RMSNorm. Native and full dynamic controls use the same hook path. Every row in Table 14 uses the same 128 held-out prompts for its task.

Table 14: Complete direction intervention results. Mean throughput is measured in tokens/s. Summed time is measured in seconds over 128 prompts. Gain and retained latency reduction are percentages relative to the matching dynamic controls.

Task	Context	Mean throughput	Gain	τ	Summed time	Retained latency reduction
GSM8K	Native z_0	131.47	0.00%	4.472	149.636	0.0%
GSM8K	Full $z_0 + \delta$	180.17	37.05%	5.988	115.310	100.0%
GSM8K	Parallel $z_0 + \delta_{\parallel}$	124.61	-5.21%	4.466	157.729	-23.6%
GSM8K	Perpendicular $z_0 + \delta_{\perp}$	177.64	35.12%	5.924	117.436	93.8%
NanoCoder	Native z_0	150.58	0.00%	5.111	191.998	0.0%
NanoCoder	Full $z_0 + \delta$	161.63	7.33%	5.552	182.780	100.0%
NanoCoder	Parallel $z_0 + \delta_{\parallel}$	150.78	0.13%	5.114	191.847	1.6%
NanoCoder	Perpendicular $z_0 + \delta_{\perp}$	170.39	13.15%	5.527	173.880	196.6%

The NanoCoder timing ratio above 100% is not evidence that the perpendicular component is functionally better than the full map. The full dynamic control saves only 9.218 seconds relative to its native dynamic control, while the perpendicular intervention saves 18.118 seconds in a separate measurement. The acceptance-length comparison is more stable and is the basis of the main-text interpretation.

The scaling probes use folded static projections. Correction strength α supplies $z_0 + \alpha\delta$. Whole-context scaling instead supplies $2(z_0 + \delta)$. Table 15 includes the native and full endpoints, every tested correction strength, and the whole-context scaling control cited in Section 6.1.

Whole-context doubling preserves the recorded acceptance length exactly on both tasks. Correction doubling does not. Relative to the full maps, mean throughput changes by +0.69% on GSM8K and -3.14% on NanoCoder. Acceptance length is a pooled ratio and does not determine each request’s block schedule, output length, or token path. The variants were timed in separate, unrepeatable passes, while scaling adds no inference operation. We therefore treat these small throughput differences as likely GPU timing variation rather than a causal speed effect.

B.2 LORA-SHIFT GEOMETRY

The geometry probe teacher-forces identical saved tokens through the base target and the LoRA-fine-tuned target. It uses 64 cached training examples per task and 16 response positions per example, for 1,024 positions. Let t be the fused-context change caused by LoRA fine-tuning and let δ be the

Table 15: Complete correction-strength and whole-context scaling results. Mean throughput is measured in tokens/s. Gain and retained latency reduction are percentages relative to the matching static native and full controls.

Task	Static intervention	Mean throughput	Gain	τ	Retained latency reduction
GSM8K	Native, $\alpha = 0$	128.90	0.00%	4.472	0.0%
GSM8K	Correction strength 0.5	177.02	37.33%	5.757	98.9%
GSM8K	Full, $\alpha = 1$	179.83	39.51%	5.988	100.0%
GSM8K	Correction strength 1.5	175.61	36.24%	5.802	89.8%
GSM8K	Correction strength 2	165.93	28.72%	5.400	72.8%
GSM8K	Whole context $\times 2$	181.07	40.47%	5.988	102.1%
NanoCoder	Native, $\alpha = 0$	150.79	0.00%	5.111	0.0%
NanoCoder	Correction strength 0.5	169.23	12.23%	5.532	90.8%
NanoCoder	Full, $\alpha = 1$	171.73	13.89%	5.552	100.0%
NanoCoder	Correction strength 1.5	163.65	8.53%	5.321	54.0%
NanoCoder	Correction strength 2	152.46	1.11%	5.032	-4.4%
NanoCoder	Whole context $\times 2$	166.34	10.32%	5.552	72.4%

map correction. The shuffled control randomly reassigns t among sampled positions while leaving every correction fixed.

Table 16: Complete LoRA-shift geometry results. Cosine similarity, radial energy fraction, and relative linearization error are dimensionless.

Task	Positions	Mean cosine	Shuffled cosine	Difference	Radial energy	RMSNorm error
GSM8K	1,024	-0.0429	-0.0413	-0.0016	25.22%	17.55%
NanoCoder	1,024	0.1248	0.1052	0.0195	34.82%	14.88%

Radial energy is the fraction of $\sum_p \|\delta_p\|^2$ parallel to $z_{0,p}$. RMSNorm error is the norm of the difference between the full normalized change and its first-order approximation, divided by the norm of the full normalized change. Neither quantity is an acceptance or speed measurement.

B.3 QWEN3-4B DRAFTER TO QWEN3-8B TARGET: FEATURE RECONSTRUCTION

The feature probe uses 64 training records and 7,090 sampled positions, split into 2,994 prompt and 4,096 response positions. Feature and context errors are normalized squared errors and are dimensionless. Context error is measured after frozen fusion and RMSNorm. Source cosine is the dimensionless cosine similarity between a mapped 8B feature and its paired 4B feature. The MSE, AUF, and decay rows are complete training recipes with different learning rates and supervision coverage.

Table 17: Complete fused-context reconstruction results. Errors are dimensionless normalized squared errors. Lower is better.

Recipe	Prompt context error	Response context error
MSE	0.216170	0.136147
AUF	1.228544	0.940413
Decay CE	1.189191	0.899731

B.4 HELD-OUT COMMON-INPUT DIAGNOSTICS

For each workload, the probe selects eight held-out responses and eight identical test positions per response, for 64 tests per variant. At each position, the frozen drafter predicts the next 15 tokens from the saved response. Full cross-entropy is measured in nats per predicted token. AUF support is the percentage of valid tokens that would receive AUF loss. Correct prefix is the mean number of consecutive correct predictions before the first error and is measured in tokens. Prompt and response

Table 18: Complete per-layer feature reconstruction results. Feature error is a dimensionless normalized squared error, and source cosine is dimensionless. P and R denote prompt and response positions. Layer numbers index the five feature streams in RelaySpec.

MSE					AUF				
Layer	P error	R error	P cosine	R cosine	Layer	P error	R error	P cosine	R cosine
0	0.060359	0.033437	0.975595	0.984793	0	2.742802	2.794326	0.195425	0.214868
1	0.191632	0.068966	0.899016	0.965129	1	2.859036	2.540209	0.112855	0.120631
2	0.192591	0.083779	0.901045	0.957394	2	21.592222	12.066430	0.141729	0.175011
3	0.191474	0.110141	0.902611	0.944060	3	6.882195	4.716092	0.144134	0.226237
4	0.180501	0.090359	0.909023	0.954783	4	3.814032	3.408715	0.190191	0.266906

Decay CE				
Layer	P error	R error	P cosine	R cosine
0	2.710681	2.753600	0.201154	0.220411
1	2.827380	2.522013	0.119284	0.126873
2	21.544376	12.034999	0.145896	0.179880
3	6.830830	4.675968	0.151039	0.234286
4	3.730459	3.298069	0.201127	0.278744

errors are dimensionless normalized context errors. These offline prefixes are not live acceptance lengths.

Table 19: Complete held-out MATH diagnostics. Cross-entropy is in nats/token, AUF support is a percentage, correct prefix is in tokens, and context errors are dimensionless.

Variant	Cross-entropy	AUF support	Correct prefix	Prompt error	Response error
MSE	1.6939	51.81%	6.7656	0.1821	0.1236
Decay midpoint	1.8043	49.79%	6.4844	0.7590	0.4928
Decay CE	1.9534	47.13%	6.0625	1.3852	0.9058
Warm unseen 4k	2.0634	52.45%	6.9219	0.3910	0.3867
Warm seen 4k	2.0695	50.64%	6.6094	0.3947	0.3827
AUF midpoint	2.1282	48.51%	6.2812	0.7768	0.5110
Warm unseen 8k	2.1507	54.47%	7.2188	0.4347	0.4084
Warm seen 8k	2.1760	56.49%	7.5625	0.4362	0.4088
AUF	2.6028	45.74%	5.8438	1.4131	0.9333
Random initial	8.6261	6.81%	0.0000	4.4041	4.9689

Table 20: Complete held-out GSM8K diagnostics. Units match Table 19.

Variant	Cross-entropy	AUF support	Correct prefix	Prompt error	Response error
Decay midpoint	1.5407	55.01%	7.1875	0.7967	0.4933
Decay CE	1.6564	50.70%	6.5312	1.4673	0.8850
Warm seen 4k	1.7356	50.91%	6.5625	0.4074	0.3686
Warm seen 8k	1.7586	57.48%	7.5781	0.4506	0.3911
Warm unseen 8k	1.7793	55.65%	7.3125	0.4535	0.3911
MSE	1.7817	53.07%	6.8750	0.2026	0.1146
Warm unseen 4k	1.8745	55.65%	7.2812	0.4053	0.3692
AUF midpoint	1.9572	55.76%	7.3125	0.8212	0.5098
AUF	2.0566	54.14%	7.0781	1.5231	0.9074
Random initial	9.6898	7.00%	0.0156	4.0335	4.6817

Table 21: Complete held-out Code diagnostics. Units match Table 19.

Variant	Cross-entropy	AUF support	Correct prefix	Prompt error	Response error
MSE	2.6039	40.46%	4.8750	0.3942	0.2864
Warm unseen 4k	3.4776	32.24%	3.6719	0.5446	0.4510
Warm unseen 8k	3.5000	32.79%	3.7344	0.5785	0.4877
Warm seen 8k	3.5864	32.02%	3.6250	0.5754	0.4818
Warm seen 4k	3.6100	33.77%	3.8906	0.5456	0.4543
Decay midpoint	3.7045	25.33%	2.6406	0.9133	0.6970
AUF midpoint	4.1200	28.62%	3.1250	0.9301	0.7159
Decay CE	4.6468	17.98%	1.5625	1.5436	1.2464
AUF	5.5081	17.00%	1.4219	1.5820	1.2868
Random initial	9.8854	7.24%	0.0312	3.7889	4.6308

Table 22: Complete held-out Chat diagnostics. Units match Table 19.

Variant	Cross-entropy	AUF support	Correct prefix	Prompt error	Response error
MSE	3.8074	28.50%	2.9062	0.3017	0.3255
Warm unseen 8k	4.1924	26.64%	2.6406	0.4872	0.4895
Warm seen 4k	4.2015	26.99%	2.6719	0.4507	0.4547
Warm seen 8k	4.2630	24.65%	2.3594	0.4868	0.4873
Warm unseen 4k	4.2904	24.42%	2.3281	0.4504	0.4519
Decay midpoint	4.5984	21.03%	1.8750	0.9437	0.7632
AUF midpoint	4.9769	22.78%	2.1094	0.9668	0.7841
Decay CE	5.6398	16.36%	1.2188	1.7617	1.3749
AUF	6.2094	13.55%	0.8438	1.8059	1.4210
Random initial	10.7415	7.48%	0.0000	4.9159	4.0375

B.5 COMPLETE LIVE MIDPOINT INTERVENTION

The live midpoint probe asks whether moving a token-trained interface toward a reconstructed interface changes decoding without further optimization. For each of the five feature streams, it averages the AUF or decay-CE map with the corresponding MSE map, using equal weight. The target, frozen drafter, prompts, and decoding settings do not change, and the midpoint is not retrained. Each parent, midpoint, and MSE endpoint uses 128 prompts on Code or MATH. Table 23 includes both token-trained parents and the common MSE endpoint for each workload. Mean and pooled throughput are measured in tokens/s. Acceptance length is measured in tokens per verification step. Total time is measured in seconds. Native and parent comparisons are shown with multiplication signs.

Latency reduction is a percentage relative to the named token-trained parent. Endpoints were timed in separate passes.

Table 23: All parent, midpoint, and MSE endpoints in the live interpolation probe.

Task	Parent	Variant	Mean throughput	Pooled throughput	Native ratio	Parent ratio	Total time	Latency reduction	τ
Code	AUF	MSE	126.18	124.01	0.8322×	2.1618×	569.96	50.08%	4.6912
Code	AUF	Midpoint	81.64	85.41	0.5384×	1.3986×	827.50	27.53%	3.2512
Code	AUF	Parent	58.37	61.90	0.3850×	1.0000×	1141.85	0.00%	2.3456
Code	Decay CE	MSE	126.18	124.01	0.8322×	2.0498×	569.96	47.75%	4.6912
Code	Decay CE	Midpoint	84.68	88.03	0.5585×	1.3756×	802.88	26.39%	3.3368
Code	Decay CE	Parent	61.56	64.80	0.4060×	1.0000×	1090.76	0.00%	2.4489
MATH	AUF	MSE	219.48	216.72	0.9986×	1.1300×	406.20	13.83%	7.9776
MATH	AUF	Midpoint	206.81	201.87	0.9410×	1.0648×	436.09	7.48%	7.5624
MATH	AUF	Parent	194.22	186.76	0.8837×	1.0000×	471.37	0.00%	6.8923
MATH	Decay CE	MSE	219.48	216.72	0.9986×	1.1273×	406.20	13.39%	7.9776
MATH	Decay CE	Midpoint	208.80	203.73	0.9500×	1.0725×	432.11	7.86%	7.5748
MATH	Decay CE	Parent	194.69	187.71	0.8858×	1.0000×	468.98	0.00%	6.9474

On Code, the AUF midpoint increases mean throughput from 58.37 to 81.64 tokens/s and acceptance length from 2.3456 to 3.2512 tokens per verification step. The decay-CE midpoint likewise increases them from 61.56 to 84.68 tokens/s and from 2.4489 to 3.3368 tokens per step. Both MATH midpoints also improve both measurements relative to their token-trained parents. None of the four midpoints reaches the MSE endpoint. Because all five maps move together, this intervention does not identify one feature stream as the cause. It establishes improvement for these two tested workloads, not for untested workloads or arbitrary interpolation weights.

B.6 COMPLETE WARM AUF REFINEMENT RESULTS

All four refinements start from the same 8k MSE parent and run AUF for three epochs. Seen data overlap the MSE training examples. Unseen data are disjoint from them. Mean throughput is measured in tokens/s. Acceptance length is measured in tokens per verification step. Fit time is measured in minutes and includes the MSE parent and AUF stage.

Table 24: All 16 warm-refinement endpoints. Native and MSE-parent throughput ratios use multi-precision signs. Latency reduction is a percentage relative to native.

Task	AUF data	Native ratio	MSE-parent ratio	Mean throughput	Latency reduction	τ	Fit time
Chat	Unseen 4k	0.8548×	0.9624×	76.51	−17.33%	2.7495	32.39
Chat	Seen 8k	0.8446×	0.9509×	75.59	−19.06%	2.7038	45.32
Chat	Unseen 8k	0.8431×	0.9492×	75.46	−19.32%	2.7030	47.42
Chat	Seen 4k	0.8420×	0.9480×	75.36	−18.58%	2.7403	31.52
Code	Unseen 4k	0.7417×	0.8913×	112.46	−26.52%	4.3499	32.39
Code	Seen 4k	0.7273×	0.8740×	110.28	−27.86%	4.2769	31.52
Code	Unseen 8k	0.7051×	0.8473×	106.92	−31.27%	4.1818	47.42
Code	Seen 8k	0.7043×	0.8463×	106.78	−32.01%	4.1525	45.32
GSM8K	Unseen 8k	1.0606×	1.0874×	188.72	5.53%	6.6218	47.42
GSM8K	Seen 8k	1.0431×	1.0696×	185.62	3.50%	6.5891	45.32
GSM8K	Unseen 4k	1.0413×	1.0677×	185.30	4.01%	6.5340	32.39
GSM8K	Seen 4k	1.0339×	1.0601×	183.97	3.40%	6.4951	31.52
MATH	Seen 8k	0.9927×	0.9941×	218.18	−1.32%	7.8662	45.32
MATH	Unseen 8k	0.9911×	0.9925×	217.82	−1.83%	7.8410	47.42
MATH	Seen 4k	0.9717×	0.9731×	213.57	−3.20%	7.8145	31.52
MATH	Unseen 4k	0.9694×	0.9707×	213.05	−3.42%	7.7785	32.39

The warm runs improve GSM8K and weaken MATH, Code, and Chat relative to the MSE parent. This pattern is consistent with workload-specific refinement, but it does not establish overfitting because matched training and held-out learning curves were not recorded.

B.7 RECTANGULAR-IDENTITY INITIALIZATION

This probe repeats the T1 8k MSE, AUF, and decaying-cross-entropy fits from a rectangular identity. Each map initially copies the first 2,560 coordinates of the 4,096-dimensional 8B feature and sets the remaining input directions to zero. Corresponding coordinates need not have the same meaning in the two models. All three retries use 8,192 training examples, three epochs, and learning rate 10^{-3} . The fusion projection, normalization, embeddings, vocabulary head, and drafter transformer remain frozen.

For MSE, the original and retry both use learning rate 10^{-3} , so their only planned recipe change is map initialization. The original AUF and decay fits use Xavier initialization and learning rate 10^{-4} . Their retries change both initialization and learning rate. Their improvements cannot be assigned to either change alone.

Table 25: Complete rectangular-identity endpoints. Mean throughput is measured in tokens/s. Native throughput ratios are shown with multiplication signs. Each workload uses 128 prompts.

Workload	Recipe	Mean throughput	Native ratio	τ
MATH	MSE	213.70	$0.9723\times$	7.983
MATH	AUF	206.16	$0.9380\times$	7.326
MATH	Decay CE	203.74	$0.9270\times$	7.348
GSM8K	MSE	166.70	$0.9368\times$	6.217
GSM8K	AUF	175.35	$0.9854\times$	6.166
GSM8K	Decay CE	169.72	$0.9538\times$	6.073
Code	MSE	119.49	$0.7881\times$	4.569
Code	AUF	73.10	$0.4821\times$	2.931
Code	Decay CE	76.02	$0.5014\times$	3.042
Chat	MSE	76.13	$0.8506\times$	2.747
Chat	AUF	60.77	$0.6790\times$	2.163
Chat	Decay CE	61.11	$0.6828\times$	2.183

Table 26: Complete comparison with the original Xavier-initialized fits. Throughput columns are measured in tokens/s. Changes are percentages relative to the original fit.

Workload	Recipe	Original throughput	Identity throughput	Change	Original τ	Identity τ	Change
MATH	MSE	218.75	213.70	-2.31%	7.973	7.983	+0.14%
MATH	AUF	194.22	206.16	+6.15%	6.892	7.326	+6.29%
MATH	Decay CE	194.69	203.74	+4.65%	6.947	7.348	+5.77%
GSM8K	MSE	172.35	166.70	-3.28%	6.204	6.217	+0.21%
GSM8K	AUF	162.05	175.35	+8.21%	5.677	6.166	+8.60%
GSM8K	Decay CE	163.47	169.72	+3.82%	5.693	6.073	+6.68%
Code	MSE	125.98	119.49	-5.15%	4.690	4.569	-2.58%
Code	AUF	58.37	73.10	+25.24%	2.346	2.931	+24.94%
Code	Decay CE	61.56	76.02	+23.49%	2.449	3.042	+24.20%
Chat	MSE	78.31	76.13	-2.78%	2.802	2.747	-1.94%
Chat	AUF	49.93	60.77	+21.73%	1.767	2.163	+22.41%
Chat	Decay CE	51.31	61.11	+19.09%	1.812	2.183	+20.52%

Table 27: Rectangular-identity fitting cost. Wall time is measured in minutes. GPU time is measured in GPU-hours on two GPUs.

Recipe	Wall time	GPU time
MSE	11.11	0.370
AUF	30.14	1.005
Decay CE	29.54	0.985

MSE retains the highest acceptance length on all four workloads and the highest measured throughput on MATH, Code, and Chat. AUF is faster on GSM8K in this single pass despite having a slightly lower acceptance length. There are no repeated timing trials, so this does not establish a stable throughput advantage or timing noise. The MSE initializer-only comparison supports retaining the selected Xavier initialization. The token-recipe retries show that their original initialization and learning rate are not the only settings under which they can learn, but do not isolate which change helped.

B.8 ADDITIONAL COMPLETED PROBES NOT USED IN THE MAIN ANALYSIS

The following completed probes informed the decision to keep the main analysis narrow. They are reported here so negative and secondary results remain visible.

B.8.1 REMOVING ONE MAP CORRECTION

Removing map i replaces W_i with the identity and leaves its native feature stream and fusion block in place. Map indices 0 through 4 correspond to target layers 1, 9, 17, 25, and 33. Throughput is measured in tokens/s. Acceptance length is measured in tokens per verification step.

Table 28: Complete single-map removal results. Gain and retained latency reduction are percentages relative to the static controls.

Task	Removed map	Mean throughput	Gain	τ	Retained latency reduction
GSM8K	0	174.69	35.52%	5.982	88.6%
GSM8K	1	178.72	38.65%	5.979	97.9%
GSM8K	2	184.68	43.27%	6.006	109.6%
GSM8K	3	176.18	36.68%	5.908	94.9%
GSM8K	4	171.68	33.18%	5.833	86.4%
NanoCoder	0	167.18	10.87%	5.577	74.9%
NanoCoder	1	170.44	13.03%	5.569	90.6%
NanoCoder	2	170.78	13.26%	5.587	92.9%
NanoCoder	3	170.51	13.08%	5.539	92.8%
NanoCoder	4	168.31	11.62%	5.447	84.9%

B.8.2 USING MAPS TRAINED FOR ANOTHER LORA-FINE-TUNED TARGET

The evaluated target, target LoRA, and prompts remain fixed while the complete five-map bank is replaced. No completed KiCad destination comparison is available.

Table 29: Complete foreign-bank results. Throughput is measured in tokens/s. Gain and retained latency reduction are percentages relative to the destination task’s native and full controls.

Destination	Source bank	Mean throughput	Gain	τ	Retained latency reduction
GSM8K	KiCad	127.63	-0.99%	4.430	-2.5%
GSM8K	NanoCoder	129.72	0.63%	4.565	4.2%
NanoCoder	GSM8K	148.97	-1.20%	5.016	-2.0%
NanoCoder	KiCad	148.12	-1.77%	4.990	-11.1%

B.8.3 LOW-RANK CORRECTION PROBES

To test how many singular directions of the learned change matter, we decompose each correction $W_i - I$, retain its leading r singular directions, and add the truncated correction back to the identity map. This preserves the original interface where the discarded correction has no effect. For comparison, whole-map truncation decomposes W_i itself and removes most of that identity path. Rank r applies separately to each of the five maps. Tables 30 and 31 give the first pass and repeated timing with their matching controls. Retained latency reduction is $(T_{\text{native}} - T_r)/(T_{\text{native}} - T_{\text{full}})$, where T is summed request latency for the same prompt cohort and timing pass. All exports use a dense folded projection, so these results test the correction’s structure rather than low-rank inference speed.

Table 30: Complete first-pass low-rank results. Throughput is measured in tokens/s. Gain and retained latency reduction are percentages relative to static controls.

Task	Truncated object	Rank	Mean throughput	Gain	τ	Retained latency reduction
GSM8K	Whole map	1	34.16	-73.50%	1.145	-1142.5%
GSM8K	Whole map	2	34.50	-73.24%	1.165	-1125.0%
GSM8K	Correction	1	143.26	11.14%	4.913	36.9%
GSM8K	Correction	2	148.99	15.58%	5.072	49.6%
GSM8K	Correction	64	172.81	34.06%	5.931	87.6%
GSM8K	Correction	128	179.10	38.94%	5.928	99.2%
GSM8K	Correction	256	172.59	33.89%	5.964	87.0%
NanoCoder	Whole map	1	31.06	-79.40%	1.057	-3484.4%
NanoCoder	Whole map	2	33.06	-78.08%	1.076	-3248.1%
NanoCoder	Correction	1	157.11	4.19%	5.239	38.8%
NanoCoder	Correction	2	151.77	0.65%	5.259	2.1%
NanoCoder	Correction	128	163.60	8.50%	5.492	60.1%
NanoCoder	Correction	256	168.04	11.44%	5.521	83.1%
NanoCoder	Correction	512	167.20	10.89%	5.528	76.2%
NanoCoder	Correction	1,024	168.84	11.97%	5.546	84.5%
NanoCoder	Correction	2,048	166.49	10.41%	5.553	72.3%

Table 31: Complete repeated timing results for selected correction ranks. Throughput is measured in tokens/s. Gain and retained latency reduction are percentages relative to the native and full controls from the same repeat.

Task and repeat	Variant	Mean throughput	Gain	τ	Retained latency reduction
GSM8K, repeat 0	Native	124.91	0.00%	4.472	0.0%
GSM8K, repeat 0	Full	176.03	40.92%	5.988	100.0%
GSM8K, repeat 0	Correction rank 128	172.81	38.35%	5.928	95.1%
GSM8K, repeat 0	Correction rank 256	180.10	44.18%	5.964	106.7%
GSM8K, repeat 1	Native	127.34	0.00%	4.472	0.0%
GSM8K, repeat 1	Full	184.00	44.50%	5.988	100.0%
GSM8K, repeat 1	Correction rank 128	183.07	43.76%	5.928	99.6%
GSM8K, repeat 1	Correction rank 256	176.10	38.29%	5.964	88.6%
NanoCoder, repeat 0	Native	152.44	0.00%	5.111	0.0%
NanoCoder, repeat 0	Full	171.03	12.20%	5.552	100.0%
NanoCoder, repeat 0	Correction rank 256	163.49	7.25%	5.521	57.8%
NanoCoder, repeat 0	Correction rank 512	166.52	9.24%	5.528	72.4%
NanoCoder, repeat 1	Native	157.04	0.00%	5.111	0.0%
NanoCoder, repeat 1	Full	172.93	10.12%	5.552	100.0%
NanoCoder, repeat 1	Correction rank 256	171.45	9.18%	5.521	89.3%
NanoCoder, repeat 1	Correction rank 512	166.80	6.22%	5.528	52.1%